



SECP3623 DATABASE PROGRAMMING

TOPIC 7

STORED PROCEDURES AND FUNCTIONS

LECTURE LEARNING OUTCOME

By the end of this lecture, students should be able to:

- Understand what stored procedures and functions are in MySQL.
- Create and execute stored procedures with parameters.
- Use variables, IF/CASE conditions, and loops within stored procedures.
- Develop and run stored functions that return values.
- Distinguish between procedures and functions and explain when to use each.

STORED PROCEDURES

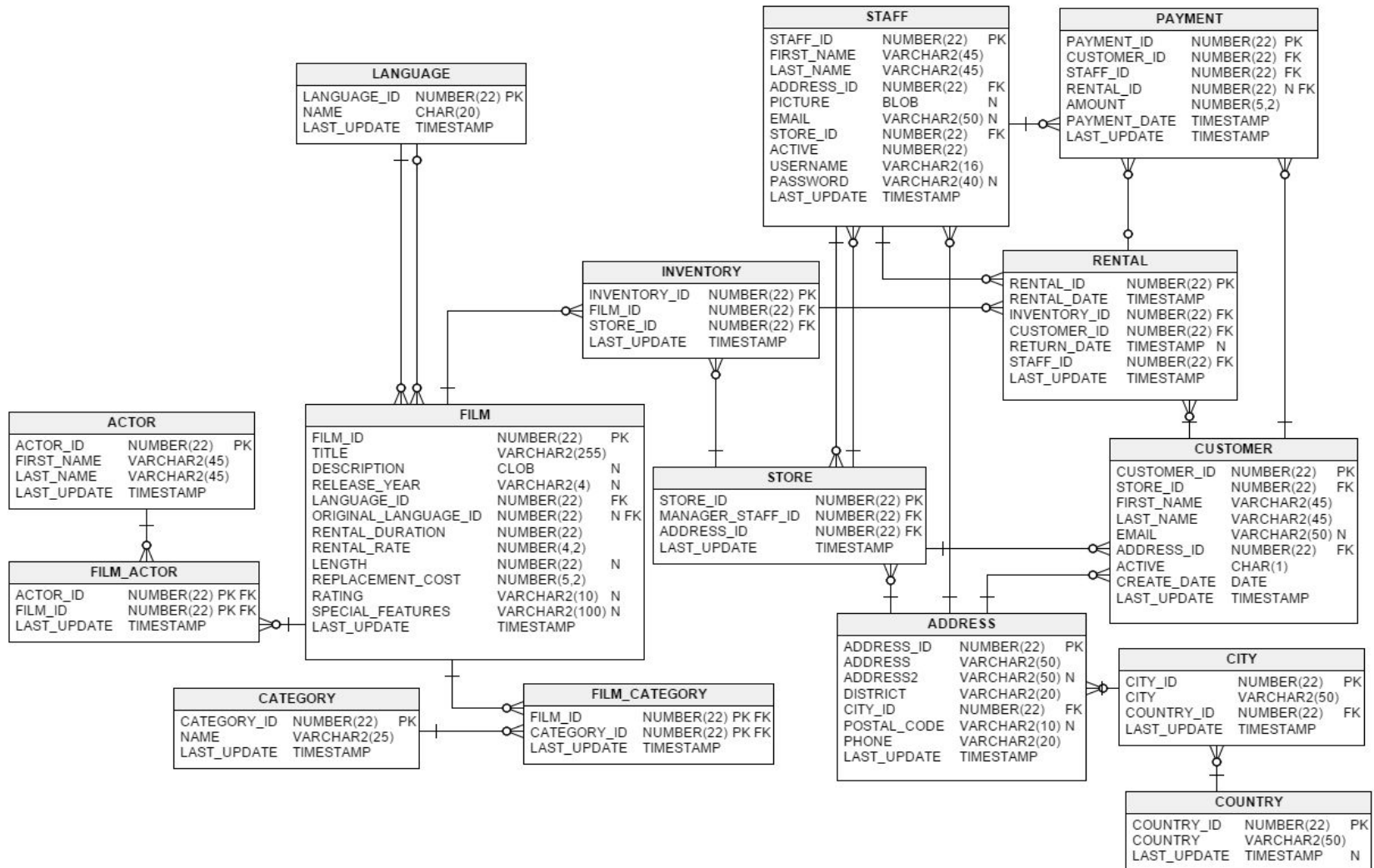
- If you want to save a query on the database server for execution later, one way to do it is to use a stored procedure.
- Once saved, can invoke the stored procedure using the CALL statement.
- A stored procedure can have parameters so you can pass values to it and get the result back. For example, you can have a stored procedure that returns customers by country and city. In this case, the country and city are parameters of the stored procedure.
- A stored procedure may contain control flow statements such as IF, CASE and loop
- A stored procedure can call other stored procedures or stored functions, which allows you to modularize your code.

CREATE STORED PROCEDURE

- Syntax:

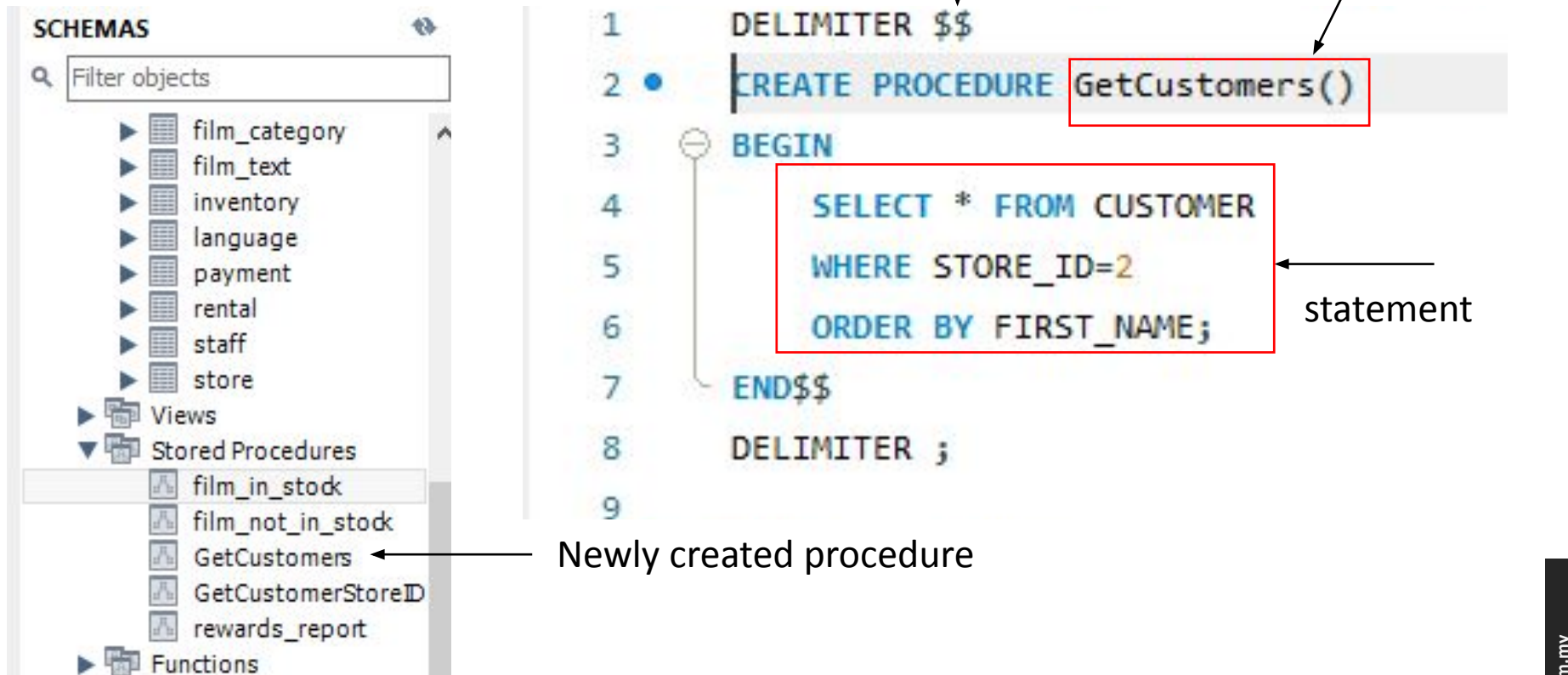
```
CREATE PROCEDURE procedure_name (parameter_list)
BEGIN
statements;
END //
```

SAKILA DATABASE



Example 1

- Using Sakila DB, create a procedure to retrieve a list of customers whose store id is 2.



The image shows a screenshot of SQL Server Enterprise Manager on the left and a SQL script on the right.

Left Panel (Schemas): The 'Schemas' pane shows the 'Sakila' database. Under 'Stored Procedures', the procedure 'GetCustomers' is highlighted, with an arrow pointing to it from the text 'Newly created procedure'.

Right Panel (SQL Script): The script is as follows:

```

1  DELIMITER $$
2  CREATE PROCEDURE GetCustomers()
3  BEGIN
4      SELECT * FROM CUSTOMER
5      WHERE STORE_ID=2
6      ORDER BY FIRST_NAME;
7  END$$
8  DELIMITER ;
9

```

Annotations on the script:

- 'Redefine delimiter **' points to line 1: `DELIMITER $$`
- 'Procedure name' points to line 2: `GetCustomers()`
- 'statement' points to the `SELECT` statement in lines 4-6.
- 'Newly created procedure' points to the `GetCustomers` procedure in the left pane.

****redefine the delimiter temporarily so that you can pass the whole stored procedure to the server as a single statement.**

Example 1 (continue)

- To execute the procedure GetCustomers(), use CALL statement.

29
30 • `call GetCustomers();`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date
▶	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:3
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:3
	217	2	AGNES	BISHOP	AGNES.BISHOP@sakilacustomer.org	221	1	2006-02-14 22:04:3
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:3
	454	2	ALEX	GRESHAM	ALEX.GRESHAM@sakilacustomer.org	459	1	2006-02-14 22:04:3
	439	2	ALEXANDER	FENNELL	ALEXANDER.FENNELL@sakilacustomer.org	444	1	2006-02-14 22:04:3

Result 1 x

Read Only

Variables In Stored Procedure

- A variable is a named data object whose value can change during the stored procedure execution.
- Use variables to hold immediate result inside stored procedures
 - These variables are **local to the stored procedure**.
- Variable must be declared before used.

```
DECLARE variable_name datatype(size) [DEFAULT  
default_value];
```

- Can declare two or more variables that share the same data type using a single **DECLARE** statement
- A user defined variable can hold a single value only

Parameters in Stored Procedure

- 3 modes of parameters: **IN** **OUT** **INOUT**
- **IN** is the default mode
 - When you define an **IN** parameter in a stored procedure, the calling program has to pass an argument to the stored procedure
 - The value of an **IN** parameter is protected
 - even the value of the **IN** parameter is changed inside the stored procedure, its original value is retained after the stored procedure ends. In other words, the stored procedure only works on the copy of the **IN** parameter.
- The value of an **OUT** parameter can be changed inside the stored procedure, and its new value is passed back to the calling program.
 - stored procedure cannot access the initial value of the **OUT** parameter when it starts.
- An **INOUT** parameter is a combination of **IN** and **OUT** parameters
 - the calling program may pass the argument, and the stored procedure can modify the **INOUT** parameter, and pass the new value back to the calling program.

Example 2: CREATE PROCEDURE with variables and parameter

Parameter IN and OUT

```
8
9  Delimiter $$
10 • CREATE PROCEDURE GetCustomerStoreID1 (in storeID int, out totCust int)
11 BEGIN
12     DECLARE newTotCust INT;
13     SELECT *
14     FROM CUSTOMER
15     WHERE STORE_ID = storeID;
16
17     select count(*) |
18     INTO newTotCust
19     from customer
20     where store_id = storeID;
21
22     set totCust = newTotCust;
23 END$$
24 delimiter ;
```

Declare variable in procedure

Assign a value from a SQL statement to a variable

Assign a value to OUT parameter

Example: Calling a Procedure

- Calling a procedure using **CALL** statement. Include argument in the **CALL** statement

argument



```
call GetCustomerStoreID1 (2, @totalCust);  
select @totalCust;
```

Conditional statement: IF Statement, CASE statement

- **IF** statement has three forms: simple **IF-THEN** statement, **IF-THEN-ELSE** statement, and **IF-THEN-ELSEIF- ELSE** statement.

```
IF condition THEN  
    statements;  
END IF;
```

IF ... THEN .. END IF

IF ... THEN .. ELSE ... END IF

```
IF condition THEN  
    statements;  
ELSE  
    else-statements;  
END IF;
```

Conditional statement: IF Statement, CASE statement

- Simple **CASE** statement:

```
CASE case_value
    WHEN when_value1 THEN statements
    WHEN when_value2 THEN statements ...
    [ELSE else-statements]
END CASE;
```

- Search **CASE** statement:

```
CASE
    WHEN search_condition1 THEN statements
    WHEN search_condition1 THEN statements ...
    [ELSE else-statements]
END CASE;
```

Example 3. IF statement

```

DELIMITER $$
create procedure star (in countryID int, out starPriv int)
begin
    declare cntCity int;
    select count(city_id)
        into cntCity
        from city
        where country_id = countryID;
    if cntCity >= 9 then
        set starPriv=5;
    elseif cntCity>4 and cntCity<9 then
        set starPriv=3;
    else
        set starPriv=1;
    end if;
end$$
  
```

Calling program:

```

35 • set @country=3;
36 • call star(@country, @starPrivilege);
37 • select @country, @starPrivilege;
38
39
40
  
```

Result Grid		Filter Rows:	Export:
@country	@starPrivilege		
3	1		

IF-ELSEIF

Practice

- Rewrite the procedure **STAR** using CASE conditional statement.

Stored Function

- A stored function is a special kind stored program that returns a single value.
- Can use stored functions to encapsulate common formulas or business rules that are reusable among SQL statements or stored programs.

- Stored Function Syntax:

```
DELIMITER $$

CREATE FUNCTION function_name(
    param1,
    param2,...
)
RETURNS datatype
[NOT] DETERMINISTIC
BEGIN
    -- statements
END $$

DELIMITER ;
```


Example Stored Function

Creates a function that returns the customer level based on credit:

customers
* customerNumber
customerName
contactLastName
contactFirstName
phone
addressLine1
addressLine2
city
state
postalCode
country
salesRepEmployeeNumber
creditLimit

```
DELIMITER $$

CREATE FUNCTION CustomerLevel(
    credit DECIMAL(10,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE customerLevel VARCHAR(20);

    IF credit > 50000 THEN
        SET customerLevel = 'PLATINUM';
    ELSEIF (credit >= 50000 AND
        credit <= 10000) THEN
        SET customerLevel = 'GOLD';
    ELSEIF credit < 10000 THEN
        SET customerLevel = 'SILVER';
    END IF;

    -- return the customer level
    RETURN (customerLevel);
END$$
DELIMITER ;
```

Example Stored Function

A SQL statement that uses the CustomerLevel function:

```
SELECT
    customerName,
    CustomerLevel(creditLimit)
FROM
    customers
ORDER BY
    customerName;
```

	customerName	CustomerLevel(creditLimit)
▶	Alpha Cognac	PLATINUM
	American Souvenirs Inc	SILVER
	Amica Models & Co.	PLATINUM
	ANG Resellers	SILVER
	Anna's Decorations, Ltd	PLATINUM
	Anton Designs, Ltd.	SILVER
	Asian Shopping Network, Co	SILVER
	Asian Treasures, Inc.	SILVER
	Atelier graphique	GOLD
	Australian Collectables, Ltd	PLATINUM
	Australian Collectors, Co.	PLATINUM

Example Stored Function

A stored procedure
GetCustomerLevel
that calls
the CustomerLevel
stored function:

```
DELIMITER $$

CREATE PROCEDURE GetCustomerLevel(
    IN customerNo INT,
    OUT customerLevel VARCHAR(20)
)
BEGIN

    DECLARE credit DEC(10,2) DEFAULT 0;

    -- get credit limit of a customer
    SELECT
        creditLimit
    INTO credit
    FROM customers
    WHERE
        customerNumber = customerNo;

    -- call the function
    SET customerLevel = CustomerLevel(credit);
END$$

DELIMITER ;
```

```
CALL GetCustomerLevel(-131,@customerLevel);
SELECT @customerLevel;
```



SECP3623 DATABASE PROGRAMMING

TOPIC 7

STORED PROCEDURES AND FUNCTIONS WITH PL/SQL (PART II)

Loop in MySQL

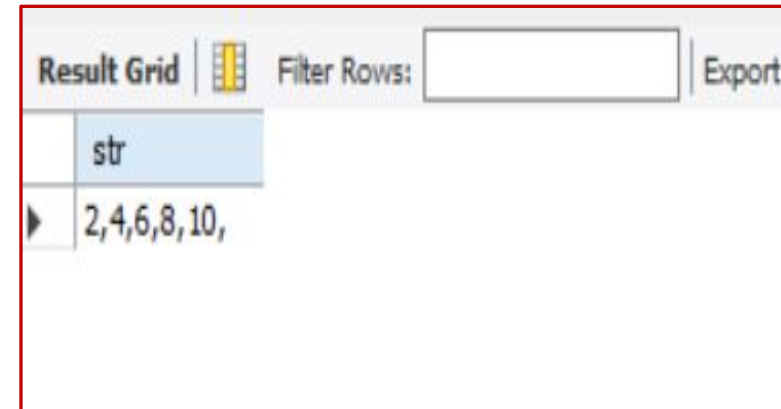
- **LOOP** statement allows you to execute one or more statements repeatedly.
- Syntax:

```
[begin-label:] LOOP  
...  
-- terminate the loop  
IF condition THEN  
    LEAVE [label];  
END IF;  
...  
END LOOP [end-label];
```

Example: LOOP in Stored Procedure

```
DROP PROCEDURE IF EXISTS LoopDemo;
DELIMITER $$
CREATE PROCEDURE LoopDemo()
BEGIN
    DECLARE x INT;
    DECLARE str VARCHAR(255);
    SET x = 1;
    SET str = "";
    loop_label: LOOP
        IF x > 10 THEN
            LEAVE loop_label;
        END IF;
        SET x = x + 1;
        IF (x mod 2) THEN
            ITERATE loop_label;
        ELSE
            SET str = CONCAT (str, ',');
        END IF;
        END LOOP loop_label;
        SELECT str;
    END$$
DELIMITER ;
```

CALL LoopDemo();



Result Grid	Filter Rows:	Export
str		
2,4,6,8,10,		

What is the purpose of these two IF s
What is **LEAVE** and **ITERATE** for?

WHILE loop in MYSQL

- The **WHILE** loop is a loop statement that executes a block of code repeatedly as long as a condition is true.

```
[begin_label:] WHILE search_condition DO  
    statement_list  
END WHILE [end_label]
```

Example WHILE loop

1. Create table Calendars.

```
1 • CREATE TABLE calendars(  
2     id INT AUTO_INCREMENT,  
3     fulldate DATE UNIQUE,  
4     day TINYINT NOT NULL,  
5     month TINYINT NOT NULL,  
6     quarter TINYINT NOT NULL,  
7     year INT NOT NULL,  
8     PRIMARY KEY(id)  
9 );
```

2. Create procedure that will insert values in table Calendars.

```
11 DELIMITER $$  
12  
13 • CREATE PROCEDURE InsertCalendar(dt DATE)  
14 BEGIN  
15     INSERT INTO calendars(  
16         fulldate,  
17         day,  
18         month,  
19         quarter,  
20         year  
21     )  
22     VALUES(  
23         dt,  
24         EXTRACT(DAY FROM dt),  
25         EXTRACT(MONTH FROM dt),  
26         EXTRACT(QUARTER FROM dt),  
27         EXTRACT(YEAR FROM dt)  
28     );  
29 END$$  
30  
31 DELIMITER ;
```


3. Create procedure LoadCalendars that will call procedure InsertCalendar to insert data into table Calendars

4. Display content table Calendars.

```
SELECT *
FROM calendars;
```

```

31 DELIMITER ;
32
33 DELIMITER $$
34
35 CREATE PROCEDURE LoadCalendars(
36     startDate DATE,
37     day INT
38 )
39 BEGIN
40
41     DECLARE counter INT DEFAULT 1;
42     DECLARE dt DATE DEFAULT startDate;
43
44     WHILE counter <= day DO
45         CALL InsertCalendar(dt);
46         SET counter = counter + 1;
47         SET dt = DATE_ADD(dt, INTERVAL 1 day);
48     END WHILE;
49
50 END$$
51
52 DELIMITER ;
  
```

id	fulldate	day	month	quarter	year
1	2019-01-01	1	1	1	2019
2	2019-01-02	2	1	1	2019
3	2019-01-03	3	1	1	2019
4	2019-01-04	4	1	1	2019
5	2019-01-05	5	1	1	2019
6	2019-01-06	6	1	1	2019
7	2019-01-07	7	1	1	2019
8	2019-01-08	8	1	1	2019
9	2019-01-09	9	1	1	2019
10	2019-01-10	10	1	1	2019
11	2019-01-11	11	1	1	2019
12	2019-01-12	12	1	1	2019
13	2019-01-13	13	1	1	2019

calendars 1 ×

Advantages of STORED PROCEDURES

- **Reusability:** Execute the same logic repeatedly by simply calling the stored procedure, reducing code duplication and promoting efficiency.
- **Modularization:** Break down complex tasks into manageable units, making code easier to read, maintain, and understand.
- **Security:** Control access to sensitive data by granting permissions to specific procedures instead of exposing underlying SQL statements.
- **Performance Benefits:** Pre-compilation can lead to faster execution in some cases, especially for complex operations.
- **Reduced Network Traffic:** Send only the procedure name and parameters to the database, reducing network overhead if multiple statements are involved.
- **Error Handling:** Implement robust error handling within procedures, improving

Disadvantages of STORED PROCEDURES

- **Increased Complexity:** Debugging and maintaining procedures can be more challenging compared to simple SQL statements.
- **Overuse Potential:** Avoid using procedures for basic tasks, as they might add unnecessary complexity.
- **Portability Issues:** Procedures might require adjustments when migrating between database systems due to vendor-specific syntax or features.
- **Security Considerations:** Improperly implemented procedures can introduce security vulnerabilities, so careful access control is crucial.

EXERCISE

Using the **Sakila** database, create a stored procedure named:

Procedure with INOUT Parameter and DECLARE variables. Create a stored procedure named:

CountCustomersByStore(IN storeID INT, INOUT customerCount INT)

The procedure must:

1. Declare a variable `totalCust`
2. Calculate number of customers for the given store ID (Sakila table: `customer`)
3. Assign the result into `totalCust`
4. Set the INOUT parameter `customerCount = totalCust`

Task:

Write the full procedure **and** show how to call it using the `CALL` statement.



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

***Innovating Solutions
Menginovasi Penyelesaian***